DEPI

Digital Egypt Pioneers Initiative

Programming Education Platform for Secondary Students

Project Documentation – Part I

Track: Full-Stack .Net Development
Technical Instructor: Mohamed Mahmoud Project Type: Team
Graduation Project Team Size: 5 Members
Team Leader: Abdelrahman Eslam
Ahmed Wajih
GitHub:https://github.com/El7a2ona/LearningPlatform

Submission Schedule (Part I)

Item	DEPI Industry	DEPI
Project Planning & Management	2 Oct 2026	16 Oct 2026
Literature Review	9 Oct 2026	16 Oct 2026
Requirements Gathering	9 Oct 2026	16 Oct 2026

Contents

- 1. Project Planning & Management – Project Proposal, Project Plan, Task Assignment & Roles, Risk Assessment & Mitigation Plan, KPIs
- 2. Literature Review – Review of Similar Solutions, Feedback & Evaluation, Suggested Improvements, Final Grading Criteria
- 3. Requirements Gathering – Stakeholder Analysis, User Stories & Use Cases, Functional Requirements, Non-functional Requirements
- 4. Repository & Project Information

1. Project Planning & Management

1.1 Project Proposal

The Programming Education Platform for Secondary Students is an online educational platform designed to help Egyptian secondary-school students learn programming through structured, accessible, and interactive courses. The platform connects students with teachers who provide educational content, quizzes, and assignments in an organized learning environment.

The system supports both free and paid courses, allowing students to explore course details, enroll in suitable courses, monitor their academic progress, and receive grades and feedback. It also provides teachers with tools to manage courses and assess students, teaching assistants with grading capabilities, guardians with access to their children's academic performance, and administrators with tools to manage complaints, approve paid courses, and maintain educational quality.

The project aims to provide an integrated learning environment that supports students throughout their programming education while simplifying course management and performance monitoring for all stakeholders.

Problem Statement

Learning programming can be challenging for secondary-school students due to the difficulty of finding organized learning resources, following a clear learning path, and evaluating their understanding of programming concepts. Students may also struggle to track their progress, receive timely feedback, and identify areas that require improvement.

At the same time, teachers need an efficient way to organize educational content, manage assessments, grade student submissions, and monitor individual and overall performance. Guardians may also lack a convenient way to follow their children's academic progress, while administrators need mechanisms to maintain course quality and handle complaints.

Therefore, there is a need for an integrated online platform that organizes programming education, simplifies communication between stakeholders, and provides effective tools for learning, assessment, progress tracking, and course management.

Proposed Solution

The proposed solution is to develop a web-based programming education platform that brings students, teachers, teaching assistants, guardians, and administrators together in one integrated system.

The platform will provide the following features:

- Structured learning: Organize courses into modules and lessons, with mandatory quizzes at the end of each module and optional lesson quizzes and assignments.
- Course discovery and enrollment: Allow students to browse courses, view course details and reviews, and enroll in free courses or purchase paid courses.
- Progress tracking and assessment: Record learning activity, quiz attempts, assignment submissions, grades, and feedback through student dashboards.
- Course management: Enable teachers to create and manage courses, prepare learning materials, assign teaching assistants, and evaluate student performance.
- Teaching assistant support: Provide access to assigned courses and tools for grading authorized student submissions.
- Guardian monitoring: Allow guardians to monitor their verified children's enrolled courses, progress, and grades.
- Administrative control: Support paid-course approval, complaint management, review moderation, and refund-request handling.
- Payment and financial management: Track eligible teacher earnings, apply refund periods, and manage payout holds according to course policies.

Through these features, the platform will provide a centralized environment for delivering, managing, and monitoring programming education

Objectives

The main objectives of the project are:

1. To provide secondary-school students with an organized and accessible platform for learning programming.
2. To simplify the process of discovering, selecting, and enrolling in suitable programming courses.
3. To support structured learning through modules, lessons, quizzes, and assignments.
4. To monitor students' academic progress and provide clear access to grades and feedback.
5. To help teachers create educational content, manage courses, and evaluate student performance efficiently.
6. To enable teaching assistants to support assessment and grading within their assigned courses.
7. To allow guardians to monitor their authorized children's academic progress.

8. To maintain educational quality through administrative course approval, complaint handling, and review moderation.
9. To support secure enrollment and payment processes for both free and paid courses.
10. To protect users' academic and financial information through role-based access control and reliable data management.
11. To provide dashboards and analytics that help teachers understand student and course performance

Scope

- User registration and login.
- Course browsing, course details, and course structure.
- Free and paid courses, enrollment, and payment.
- Course modules, lessons, mandatory module quizzes, optional lesson quizzes, and lesson assignments.
- Student progress tracking, assignment submission, quiz attempts, grading, grades, and feedback.
- Course reviews and ratings.
- Teacher course management and teacher dashboards.
- Teaching assistant assignment, assigned-course access, and grading.
- Guardian monitoring of linked children.
- Student complaints and administrative complaint handling.
- Administrative approval of paid courses and teacher compliance acknowledgement.
- Refund periods, delayed teacher payout, and teacher balance.
- Course lifecycle management.

Technology Stack

Area	Technology
Frontend	HTML, CSS, JS
Backend	ASP.NET
Database	SQL Server
AI	
Authentication	JWT
Version Control	Git + GitHub, feature branches + Pull Requests

1.2 Project Plan

The project runs for **9 weeks, from 4 Oct to 4 Dec 2026**. The plan covers the whole project (analysis, design, implementation, testing, and final delivery) and is independent of the individual documentation deadlines. Implementation is finished and deployed by **28 Nov**, ahead of the 30 Nov implementation deadline.

Phases

Phase	Dates	Goal
1. Planning & Setup	4 – 17 Oct (W1–W2)	Idea, scope, repository, branches, task split, data models and API contracts agreed
2. Backend Foundation	18 – 24 Oct (W3)	Authentication, core models, and core APIs working and tested in Postman
3. Core Features	25 Oct – 7 Nov (W4–W5)	Main screens and APIs built for each feature, including AI analysis and chat
4. Integration & Completion	8 – 21 Nov (W6–W7)	Frontend connected to backend, all features complete, feature freeze
5. Testing & Deployment	22 – 28 Nov (W8)	Test cases, bug fixing, deployment, README

6. Final Delivery	29 Nov – 4 Dec (W9)	Final reports, presentation, demo, rehearsal

Gantt Chart

Workstream	W1	W2	W3	W4	W5	W6	W7	W8	W9
Planning, SRS and repo setup									
Auth and role-based access									
Courses, modules and lessons									
Enrollment, access rules and student progress									
Quizzes, assignments and grading (including teaching assistants)									
Reviews, complaints and admin moderation									
Dashboards (teacher, teaching assistant, guardian)									
Paid courses, payments, refunds and payouts									
Shared UI and integration									
Testing and bug fixing									
Deployment and README									
Final reports and presentation									
Buffer & Rehearsal									

W1 = 4–10 Oct, W2 = 11–17 Oct, W3 = 18–24 Oct, W4 = 25–31 Oct, W5 = 1–7 Nov, W6 = 8–14 Nov, W7 = 15–21 Nov, W8 = 22–28 Nov, W9 = 29 Nov–4 Dec.

Milestones

#	Milestone	Date
M1	SRS baseline, repo and API contracts agreed	17 Oct
M2	Auth with roles, plus User, Course, Module and Lesson models	24 Oct
M3	Course browsing, free enrollment and teacher course creation	31 Oct
M4	Quizzes, assignments, submission and grading (teacher and teaching assistant)	7 Nov
M5	All five features integrated	14 Nov
M6	Feature freeze (all features complete)	21 Nov
M7	Code freeze and deployed version live	28 Nov
M8	Final presentation, demo, and reports ready	4 Dec

Deliverables

Deliverable	Owner	Due
GitHub repository with branches and PR history		Continuous
Backend APIs for all five features		21 Nov
Frontend for all screens		21 Nov
Gemini AI integration with fallback		14 Nov
Admin dashboard		21 Nov
Test cases, test plan, bug reports		28 Nov
API documentation		28 Nov
Deployed web app and README		28 Nov
Technical Documentation		3 Dec
User Manual		3 Dec

Deliverable	Owner	Due
Project Infographic		3 Dec
Presentation (PPT/PDF)		3 Dec
Video demo (optional)		3 Dec

Resource Allocation

Resource	Allocation
Team	5 developers; each owns one feature across frontend, backend, database, API, and integration
Version control	GitHub with feature branches. No direct push to main; every merge goes through a reviewed Pull Request
External services	SQL Server
Meetings	Weekly planning (Sunday), mid-week sync on blockers, Thursday demo and merge of reviewed PRs
Review process	At least one other developer reviews each Pull Request before merging

1.3 Task Assignment & Roles

Team Leader: Gamila Abdel Khalek. Each developer owns a complete feature from frontend to database and API integration, so nobody is assigned to frontend or backend only.

DEV	Member	Area	Main Responsibilities
DEV 1	Abdelrahman Eslam		
DEV 2	Tameer Saeed		
DEV 3	Malak Abdulhamid		

DEV	Member	Area	Main Responsibilities
DEV 4	Mohamed Ahmed		
DEV 5	Ola Sherif		

1.4 Risk Assessment & Mitigation Plan

Risk	Impact	Likelihood	Mitigation
Integration conflicts between features	High	Medium	Agree on API contracts early; use feature branches and Pull Requests; integration weeks W4–W7
Limited experience with some backend concepts	Medium	Medium	Work incrementally, review code together, test each feature before integration
AI analysis failure or inconsistent output (Gemini)	Medium	Medium	Validate AI output and provide a manual fallback for listing creation; confirm API key and quota in W2
Scope becoming too large	High	Medium	Prioritize the MVP; cut Exchange Proposal first if needed (see Scope Control)
Merge or regression issues	Medium	Medium	Require self-testing and code review before merging

Risk	Impact	Likelihood	Mitigation
Time pressure near deadlines	High	High	Track milestones weekly; feature freeze 21 Nov; deployment by 28 Nov; buffer week W9
A team member unavailable or behind schedule	High	Low	Raise delays immediately; rebalance tasks or reduce scope; shared ownership of reviews
Deployment or hosting problems	Medium	Medium	Start deployment preparation in W7; keep a local run guide in the README

1.5 Key Performance Indicators (KPIs)

KPI	Target	How It Is Measured
Feature completion	100% of MVP features complete by the 21 Nov freeze	Checklist per feature, reviewed weekly
API response time	Under 2 seconds for marketplace browsing and search	Postman / browser network tab
Search and filter accuracy	Results match the selected filters and show only active, approved listings	Test cases
Authentication and authorization	Protected routes reject unauthenticated users; admin routes reject non-admins	Test cases
AI listing autofill	AI result returned or fallback shown in 100% of attempts	Test cases with valid and failing inputs
Test coverage	Test cases written and run for every feature; critical bugs fixed before release	Test plan and bug report
System availability	Deployed app accessible during the evaluation period	Manual checks, deployment link
Repository and documentation	All source code and documents in GitHub; every merge via reviewed PR	Repository history
User adoption (pilot)	Pilot test with classmates; positive feedback on core flows	Short feedback form after deployment

2. Literature Review

2.1 Feedback & Evaluation

This section records the lecturer's assessment of the project. It is completed after the project is presented to the Technical Instructor, Mohamed Mahmoud.

Area	Lecturer Feedback	Team Response
Project idea and scope		
Planning and documentation		
Requirements		
Overall evaluation		

2.2 Suggested Improvements

Areas where the project can be enhanced, based on the team's own review (to be extended with the lecturer's suggestions):

- Improve AI features.
- Add a mobile application.

2.3 Final Grading Criteria

The official breakdown of marks across documentation, implementation, testing, and presentation is provided by the Technical Instructor. The table is completed once the grading scheme is confirmed.

Component	Marks	Related Project Deliverables
Documentation		Planning, literature review, requirements, system analysis and design, reports
Implementation		Source code, GitHub repository, deployed application, README
Testing		Test cases and test plan, automated tests, bug reports
Presentation		Presentation (PPT/PDF), infographic, optional video demo
Total		

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Needs	Influence / Interest
Students / Buyers	Find relevant items quickly; trust listings; contact sellers; request items they need	High interest, medium influence
Students / Sellers	Publish listings easily (with AI help); manage and mark items as sold; reach interested buyers	High interest, medium influence
Platform Administrators	Moderate listings; manage users, categories, and reports; view platform statistics	High interest, high influence
Technical Instructor	Clear documentation, working implementation, good engineering practices	High interest, high influence
Project Team	Clear task ownership, manageable scope, stable integration	High interest, high influence

3.2 User Stories & Use Cases User Stories

- As a student, I want to register and log in so that I can securely use my account.
- As a student, I want to browse active listings so that I can discover available items.
- As a student, I want to search and filter listings so that I can quickly find relevant products.
- As a student, I want to save listings as favorites so that I can return to them later.
- As a student, I want to create and manage my listings so that I can sell items.
- As a student, I want AI assistance when creating a listing so that product information can be filled more efficiently.
- As a student, I want to mark my item as sold so that other students know it is no longer available.
- As a student, I want to create a request for an item I need so that other students can respond.
- As a student, I want to communicate with another student so that I can discuss a listing or request.

- As an administrator, I want to moderate listings so that inappropriate or invalid content can be controlled.
- As an administrator, I want to manage users and categories so that the platform remains organized.
- As an administrator, I want to review reports so that I can act on reported content.

Use Cases

Actor	Use Case	Description
Student	Register / Login	Create an account and authenticate
Student	Browse Listings	View available marketplace items
Student	Search & Filter	Find listings using search and marketplace filters
Student	Manage Favorites	Save or remove listings from favorites
Student	Manage Listings	Create, edit, view, and delete owned listings; mark as sold
Student	Use AI Listing Assistant	Upload an image, receive AI analysis, review and publish
Student	Create Request	Publish a request for an item needed
Student	Chat	Communicate with other students
Student	Propose Exchange (optional)	Propose an exchange; the other student accepts or declines
Admin	Manage Users	Search, filter, block, and unblock users
Admin	Moderate Listings	Approve or reject listings
Admin	Manage Reports	Review and manage reported content
Admin	Manage Categories	Create, update, and delete categories

Sample Use Case Description: Create Listing with AI Assistance

Field	Description
Actor	Student (authenticated)
Precondition	The student is logged in
Main flow	1. Student opens Add Item and uploads an image. 2. The system sends the image for AI analysis. 3. The system returns structured data and fills the form. 4. The student reviews and edits the data. 5. The student publishes; the listing awaits approval.
Alternative flow	If AI analysis fails, the system shows a message and the student fills the form manually
Postcondition	The listing is saved and visible after admin approval

3.3 Functional Requirements

FR-01: The system shall allow students to register and log in.

FR-02: The system shall authenticate users using JWT-based authentication.

FR-03: The system shall support role-based authorization for administrative operations.

FR-04: The system shall allow users to create, edit, view, and delete their listings according to ownership rules.

FR-05: The system shall support image upload for listings.

FR-06: The system shall provide AI-assisted listing analysis and structured data extraction, with a manual fallback if analysis fails.

FR-07: The system shall allow users to browse active and approved listings.

FR-08: The system shall provide search by listing information.

FR-09: The system shall provide filtering by category, price range, condition, faculty, and department.

FR-10: The system shall support sorting and pagination of marketplace results.

FR-11: The system shall allow authenticated users to add and remove favorites.

FR-12: The system shall allow students to create, view, edit, and delete requests according to permissions.

FR-13: The system shall support conversations and messages between users.

FR-14: The system shall provide administrators with user management, including block and unblock.

FR-15: The system shall support listing approval/rejection, reports, and category management.

FR-16: The system shall allow a seller to mark a listing as sold.

FR-17: The system shall display admin statistics on the admin dashboard.

FR-18 (optional): The system shall support exchange proposals that can be accepted or declined.

3.4 Non-functional Requirements

Category	Requirement
Security	Authentication, role-based authorization, bcrypt password hashing, input validation, and protected operations
Performance	Efficient search, filtering, pagination, and API responses; marketplace pages respond within about 2 seconds
Usability	Clear navigation, loading states, empty states, and understandable success/error messages
Reliability	Graceful error handling and fallback behavior when the external AI service fails
Maintainability	Modular code, reusable components, meaningful naming, and clear project structure
Scalability	Modular architecture that allows adding marketplace features later
Compatibility	Responsive web interface for common desktop and mobile screen sizes
Localization	Arabic/English support with RTL/LTR interface direction, and light/dark mode

4. Repository & Project Information

GitHub Repository: https://github.com/SWESiham/UniSwap

The repository contains the project source code and project documentation. The team maintains a feature-branch workflow and uses Pull Requests for controlled integration.

Next Documentation Phase

The next phase expands the requirements into System Analysis & Design deliverables: software architecture, ER diagram and schema, use case diagram, DFD, sequence, activity, state and class diagrams, UI/UX mockups, component and deployment views, API documentation, and the testing plan.